

Compilation

Fiche de révision — Cours magistral, partie 2

Analyse syntaxique ascendante (LR(1), LALR(1)) • Traitement des erreurs • Génération d'analyseurs
• Analyse sémantique • Production de code

Plan du document

- **1.** Analyse ascendante : rappels et conflits SLR(1)
- **2.** Analyse LR(1) (règles pointées étendues, saturation, tables)
- **3.** Analyse LALR(1) (fusion des états) et bilan des grammaires
- **4.** Traitement des erreurs (panique, syntagme, local, global)
- **5.** Génération d'analyseurs syntaxiques (JavaCC, Yacc/Bison, JCup)
- **6.** Analyse sémantique (DDS, attributs, graphe de dépendances, table des symboles, typage)
- **7.** Production de code (code à 3 adresses, optimisations, registres, code cible)
- **8.** Récapitulatif et pièges classiques

*Convention : les symboles non terminaux sont en majuscules ($S, G, D...$), les terminaux en minuscules ou symboles ($i, =, *$). Le point « . » dans une règle pointée marque la position courante de l'analyseur ; « \$ » est la fin de mot ; « ϵ » désigne le mot vide.*

1. Analyse ascendante : rappels et conflits SLR(1)

1.1 Principe d'un analyseur LR(k)

- **Left-to-right scanning** : on lit l'entrée de gauche à droite.
- **Rightmost derivation in reverse** : on construit une dérivation droite à l'envers, c'est-à-dire qu'on part du mot et on remonte vers l'axiome (par « réductions »).
- **k** : la prédiction (lookahead) utilise k lettres du mot d'entrée.
- On travaille toujours sur une **grammaire augmentée** : on ajoute une règle $S' \rightarrow S$ (S' devient le nouvel axiome). Cela donne un état « accepter » clair.

L'analyseur manipule une **pile** (états de l'automate) et deux actions principales :

- **décalage (d q)** : lire un terminal et empiler l'état q ;
- **réduction (r $X \rightarrow \beta$)** : dépiler $|\beta|$ états puis empiler le successeur donné par la table *Successeur* pour X ;
- **accepter / erreur**.

1.2 Les différentes analyses ascendantes

Analyse	Idée	Où est la prédiction ?
LR(0)	Pas de lookahead : on réduit dès qu'une règle est complète	Aucune
SLR(1)	On réduit $X \rightarrow \beta$. seulement si le terminal lu $\in$ Suivant(X)	Ensemble Suivant (global à la grammaire)
LR(1)	Chaque règle pointée porte son propre symbole de prévision	Dans les items (précis, mais beaucoup d'états)
LALR(1)	LR(1) dont on fusionne les états de même cœur	Dans les items fusionnés (autant d'états que LR(0))

1.3 Exemple fil rouge : la grammaire G2

```
(0) S' -> S
(1) S -> G = D | D
(2) G -> * D | i
(3) D -> G
```

Suivant(S') = { $\$$ } Suivant(S) = { $\$$ }
Suivant(G) = { $=$, $\$$ } Suivant(D) = { $\$$, $=$ }

G2 modélise des « affectations » de pointeurs : $*i = i$ ($G = D$, où G est la partie gauche, D la droite), ou simplement une expression D . Les mots reconnus sont par exemple i , $*i$, $i=*i$, $**i=i$.

Automate LR(0) de G2 : l'état conflictuel

Dans l'automate LR(0), l'état 4 est atteint après avoir reconnu un G depuis l'état 1. Il contient :

État 4 : $S \rightarrow G \cdot = D$ (on peut DÉCALER ' $=$ ')
 $D \rightarrow G \cdot$ (on peut RÉDUIRE $D \rightarrow G$)

Avec SLR(1), on réduit $D \rightarrow G$. sur tout terminal de $\text{Suivant}(D) = \{\$, =\}$. Or ' $=$ ' $\in$ $\text{Suivant}(D)$: en lisant ' $=$ ', on peut donc **décaler** (d 9) ET **réduire** (r $D \rightarrow G$). C'est un **conflit décalage/réduction** dans la case (état 4, ' $=$ ') de la table Action SLR(1).

À retenir

G2 n'est ni LR(0), ni SLR(1).

Cause : l'ensemble Suivant est « global » : ' $=$ ' peut suivre D dans certains contextes, mais pas dans le contexte précis de l'état 4. Il faut une prévision **contextuelle** $\rightarrow$ analyse LR(1).

2. Analyse LR(1)

2.1 Définitions

Définition

Règle pointée étendue : une règle pointée accompagnée d'un symbole de prévision $a \in (V_T \cup \{\$, \})$, de la forme $X \rightarrow \alpha.\beta$, a . On note $X \rightarrow \alpha.\beta$, $a|b$ pour abréger deux items de même cœur ayant a et b comme prévisions.

Ensemble saturé : E est saturé si, dès que $X \rightarrow \beta.Y\gamma$, $a \in E$ (Y non terminal), alors pour toute règle $Y \rightarrow \alpha$ de la grammaire, $Y \rightarrow .\alpha$, $b \in E$ pour tout $b \in \text{Premier}(\gamma a)$.

Saturation(E) : le plus petit ensemble saturé contenant E .

Lecture intuitive : « Si je m'attends à voir Y juste après le point, alors je m'attends à voir les débuts de toutes les règles de Y ; et ce qui peut suivre Y dans ce contexte est $\text{Premier}(\gamma a)$ (ce qui vient après Y dans la règle, puis la prévision héritée si γ peut disparaître). »

2.2 Automate LR(1)

- **États** = ensembles saturés de règles pointées étendues (« collections LR(1) »).
- **État initial** = $\text{Saturation}(\{S' \rightarrow .S, \$\})$.
- **Transition** depuis E par $x \in (V_T \cup V_N)$: $\text{Saturation}(\{X \rightarrow \beta x.\gamma, a \mid X \rightarrow \beta.x\gamma, a \in E\})$: on avance le point, on garde la prévision.

Exemple — saturation de $\{S' \rightarrow .S, \$\}$ sur G_2

$\text{Premier}(G) = \text{Premier}(D) = \text{Premier}(S) = \{*, i\}$.

- $S' \rightarrow .S, \$$
- $S \rightarrow .G=D, \$$ et $S \rightarrow .D, \$$ (S est suivi de la prévision $\$$)
- $G \rightarrow .*D, =$ et $G \rightarrow .i, =$ (dans $S \rightarrow .G=D, \$$, ce qui suit G est « $= D \$$ », donc $\text{Premier} = \{=\}$)
- $D \rightarrow .G, \$$ (car D vient de $S \rightarrow .D, \$$)
- $G \rightarrow .*D, \$$ et $G \rightarrow .i, \$$ (G est aussi attendu via $D \rightarrow .G, \$$)

On remarque que $G \rightarrow .*D$ a deux prévisions différentes ($=$ et $\$$) selon le contexte : c'est exactement ce que SLR(1) ne savait pas distinguer. On note $G \rightarrow .*D, =|\$$.

2.3 États de l'automate LR(1) de G_2

État	Contenu (items)	Remarque
0	$S' \rightarrow .S, \$$; $S \rightarrow .G=D, \$$; $S \rightarrow .D, \$$; $G \rightarrow .*D, = \$$; $G \rightarrow .i, = \$$; $D \rightarrow .G, \$$	initial
1	$S' \rightarrow S., \$$	accepter
2	$S \rightarrow G.=D, \$$; $D \rightarrow G., \$$	plus de conflit : la réduction $D \rightarrow G$ n'est valable que sur $\$$
3	$S \rightarrow D., \$$	réduire
4	$G \rightarrow *.D, = \$$; $D \rightarrow .G, = \$$; $G \rightarrow .*D, = \$$; $G \rightarrow .i, = \$$	
5	$G \rightarrow i., = \$$	réduire
6	$S \rightarrow G.=D, \$$; $D \rightarrow .G, \$$; $G \rightarrow .*D, \$$; $G \rightarrow .i, \$$	après '='
7	$G \rightarrow *.D., = \$$	
8	$D \rightarrow G., = \$$	
9	$S \rightarrow G=D., \$$	
10	$D \rightarrow G., \$$	même cœur que 8
11	$G \rightarrow *.D, \$$; $D \rightarrow .G, \$$; $G \rightarrow .*D, \$$; $G \rightarrow .i, \$$	même cœur que 4
12	$G \rightarrow i., \$$	même cœur que 5
13	$G \rightarrow *.D., \$$	même cœur que 7

Point clé : dans l'état 2 (venu de l'état 0 par G), $D \rightarrow G$. n'a que la prévision $\$$. Le conflit avec le décalage de '=' disparaît.

2.4 Tables LR(1)

- **Successeur** : si $\Delta(q, X) = q'$ (X non terminal), on met q' dans la case (q, X) .
- **Action** : si $\Delta(q, a) = q'$ (a terminal) $\rightarrow$ « d q' » en (q, a) ; pour une règle complète $X \rightarrow \beta$, a dans q avec $X \neq S' \rightarrow$ « r $X \rightarrow \beta$ » en (q, a) ; « accepter » en $(q, \$)$ si q contient $S' \rightarrow S, \$$; « erreur » ailleurs.
- **Différence avec SLR(1)** : la réduction est placée uniquement sur la prévision a de l'item, pas sur tout Suivant(X).

Si chaque case de Action contient **au plus une action**, la grammaire est LR(1).

Table Action LR(1) de G2 (14 états)

	*	i	=	\$
0	d4	d5		
1				accepter
2			d6	r $D \rightarrow G$
3				r $S \rightarrow D$
4	d4	d5		
5			r $G \rightarrow i$	r $G \rightarrow i$
6	d11	d12		
7			r $G \rightarrow *D$	r $G \rightarrow *D$
8			r $D \rightarrow G$	r $D \rightarrow G$
9				r $S \rightarrow G=D$
10				r $D \rightarrow G$
11	d11	d12		
12				r $G \rightarrow i$
13				r $G \rightarrow *D$

Exemple — décalage vs réduction sans ambiguïté

Dans l'état 2 : sur '=' on décale (d6) ; sur '\$' on réduit $D \rightarrow G$. Une seule action par case $\rightarrow$ pas de conflit, G2 est donc LR(1).

En revanche, G2 n'était pas SLR(1) : la différence vient uniquement de la finesse de la prévision.

À retenir

Toute grammaire SLR(1) est LR(1).

Inconvénient de LR(1) : très grand nombre d'états (14 ici contre 11 en LR(0)), donc beaucoup de mémoire.

Cause de l'explosion : un même cœur LR(0) est dupliqué selon les prévisions possibles.

3. Analyse LALR(1) (« Look-Ahead LR(1) »)

3.1 Principe

Définition

Deux états LR(1) ne différant que par les symboles de prévision ont le même **cœur LR(0)**.

Automate LALR(1) : on fusionne les états LR(1) de même cœur. L'état fusionné contient l'union de leurs règles (donc l'union des prévisions). Les transitions sont mises à jour.

Conséquence : l'automate LALR(1) a exactement **le même nombre d'états que l'automate LR(0)**.

Exemple — fusions dans G2

- 4 et 11 → état 4/11 ($G \rightarrow *.D, = | \$ \dots$)
- 5 et 12 → état 5/12 ($G \rightarrow i., = | \$$)
- 7 et 13 → état 7/13 ($G \rightarrow *.D., = | \$$)
- 8 et 10 → état 8/10 ($D \rightarrow G., = | \$$)

On passe de 14 états (LR(1)) à 10 états (LALR(1)) = les 10 états de LR(0).

3.2 Table Action LALR(1) de G2

	*	i	=	\$
0	d4/11	d5/12		
1				accepter
2			d6	r $D \rightarrow G$
3				r $S \rightarrow D$
4/11	d4/11	d5/12		
5/12			r $G \rightarrow i$	r $G \rightarrow i$
6	d4/11	d5/12		
7/13			r $G \rightarrow *.D$	r $G \rightarrow *.D$
8/10			r $D \rightarrow G$	r $D \rightarrow G$
9				r $S \rightarrow G=D$

Construction identique à LR(1), mais à partir de l'automate LALR(1). Si chaque case contient au plus une action, la grammaire est LALR(1).

3.3 Comparaison et théorie

- Toute grammaire LALR(1) est LR(1) ; il existe des grammaires LR(1) qui ne sont pas LALR(1).
- La fusion ne peut pas créer de conflit décalage/réduction nouveau, mais **peut créer des conflits réduction/réduction** (deux prévisions qui étaient séparées se retrouvent dans le même état). C'est pour cela que $LALR(1) \subset LR(1)$ strictement.
- Toute grammaire LR(k) (donc SLR(k), LALR(k)) est **non ambiguë**.
- Analyse LR(k) avec $k > 1$: trop coûteuse en pratique.
- Il existe une méthode directe pour construire les tables LALR(1) sans construire l'automate LR(1) complet (c'est ce que font Yacc, Bison, Jcup).

3.4 Hiérarchie des grammaires

$LR(0) \subset SLR(1) \subset LALR(1) \subset LR(1) \subset \dots \subset LR(k) \subset \text{non ambiguës}$

Compromis :
SLR(1) : peu d'états, mais limite la forme de la grammaire
LR(1) : le plus puissant ($k=1$), mais très coûteux en états
LALR(1) : presque aussi puissant, taille = SLR(1)

4. Traitement des erreurs

4.1 Compilateur « idéal » : deux modes

- **Mode parser** au départ.
- Détection d'une erreur → **mode erreur** : localiser et diagnostiquer (trouver la faute à son origine), corriger si possible, puis retour au mode parser.
- Objectif : continuer l'analyse pour découvrir le plus d'erreurs possible en une seule passe.

4.2 Les quatre méthodes

Méthode	Principe	Quand ?
Mode panique	Sans correction. Avec ou sans récupération (par restart ou continuation) sur un symbole de synchronisation (mot-clé error en JCup)	Simple, localisation vague
Correction au niveau du syntagme	Routines de traitement d'erreurs placées dans la table d'analyse	Erreurs de programmation classiques
Correction locale	Règles spécifiques « cas d'erreur » ajoutées à la grammaire	Erreurs courantes décrites précisément
Correction automatique globale	L'analyseur « devine » les intentions du programmeur	Trop coûteux, non utilisé

4.3 Mode panique : récupération par continuation / par restart

Soit X le non-terminal en cours d'analyse quand l'erreur est détectée.

- **Par continuation** : on clôt le sous-arbre en cours. On se resynchronise : (a) sur un terminal **final** de X (utilise last/dernier) : End pour bloc_inst,) pour liste_param... Problème : il n'existe pas toujours de marqueur de fin ; (b) sur un premier terminal **après X** (follow/suivant) : ; pour une instruction, then ou do pour une condition. Problème : risque de propagation d'erreurs sur construction récursive.
- **Par restart** : on analyse quand même des sous-arbres de l'arbre en cours. On se resynchronise sur (a) un premier symbole d'un non-terminal accessible depuis X (first/premier) : while ou for pour bloc_inst. Problème : pas toujours de symbole initial ; (b) un symbole prédécesseur (precede/last) : ; pour une instruction, = pour une expression dans une affectation.

4.4 Exemple complet : expressions arithmétiques

```
(0) E' -> E          Priorités : Prio(*) > Prio(+)  
(1) E  -> E + E      Associativité à gauche  
(2) E  -> E * E      (la grammaire est ambiguë, mais les conflits sont  
(3) E  -> ( E )      résolus par priorité/associativité dans la table LR(0))  
(4) E  -> nb
```

a) Panique par continuation sur ')' : on ajoute la règle (5) $E \rightarrow \text{error}$ {afficher("récup E sur ') '")}. Dans les états où une erreur est possible, on met « r5 » dans la colonne ')': la sous-expression fautive est vue comme un E et on reprend après la parenthèse fermante. Exemple : $(3 + * 4) + 1 \rightarrow$ l'erreur est détectée sur '*', on avale jusqu'à ')' et on poursuit avec « + 1 ».

b) Correction au niveau du syntagme : on remplace les cases « erreur » de la table par des routines e1...e4 :

Routine	États	Message	Correction
e1	0, 1, 5, 6	nb ou parenthèse ouvrante attendu (au lieu d'opérateur ou \$)	empiler un nb et aller en état 2
e2	0, 1, 3, 5, 6	parenthèse fermante en trop	ignorer cette parenthèse
e3	3, 4	opérateur ou parenthèse fermante attendu (au lieu de nb ou '(')	empiler un opérateur et calculer l'état
e4	4	opérateur ou parenthèse fermante attendue (au lieu de \$)	empiler une parenthèse fermante et aller en état 7

Exemple — e1 et e4

- Entrée 3 + * 4 : après '+', on est dans l'état 5 et on lit '*' : routine e1 → message « nb ou '(' attendu », on empile un nb fictif (état 2) et l'analyse continue.
- Entrée (3 + 4 puis \$: on est dans l'état 4 avec '\$' : routine e4 → message « ')' attendue », on empile ')' (état 7) et on réduit $E \rightarrow (E)$.

c) Correction locale : on augmente la grammaire avec des règles qui reconnaissent les erreurs courantes :

$E \rightarrow E +)$	{action erreur nombre oublié}	ex : (3 +)
$E \rightarrow E *)$	{action erreur nombre oublié}	
$E \rightarrow E E$	{action erreur opérateur oublié}	ex : 3 4

4.5 Génération d'analyseurs syntaxiques

Un générateur prend une **grammaire** et produit un **automate à pile** (l'analyseur).

- **Parsers descendants (LL)** : écrire son propre parser est facile mais long, peu robuste, difficile à modifier. Automatiquement : **JavaCC** pour Java.
- **Parsers ascendants (LR)** : idem, ou automatiquement avec la famille **Yacc** : Yacc et Bison pour C, **JCup** pour Java (ce sont des analyseurs LALR(1)).

À retenir

Analyseur syntaxique = automate à pile reconnaissant un langage algébrique : soit descendant LL (la grammaire doit être LL), soit ascendant LR (plus général au niveau des grammaires).

5. Analyse sémantique

Objectif : à partir de l'arbre syntaxique, vérifier les propriétés sémantiques **statiques** (contextuelles) du programme et produire un **arbre syntaxique décoré**. Exemples : vérification de types, analyse de pointeurs, évaluation statique d'expressions, déclaration des variables. C'est une partie très importante et complexe d'un compilateur. La syntaxe seule ne sait pas dire « cette variable n'a pas été déclarée » : c'est le rôle de la sémantique.

5.1 Définition dirigée par la syntaxe (DDS = grammaire attribuée)

Définition

Une DDS est la donnée d'une grammaire et de ses **règles sémantiques** :

- chaque symbole (terminal ou non) possède un **ensemble d'attributs** ;
- chaque règle de production possède un ensemble de règles sémantiques (instructions algorithmiques) qui calculent les valeurs des attributs des symboles de la production.

Notation : $X.a$ = attribut a du symbole X . S'il y a plusieurs X dans une production : $X_{(0)}$ s'il est en partie gauche, sinon $X_{(1)} \dots X_{(n)}$ de gauche à droite.

Exemple — calcul d'une expression avec niveau de parenthésage

Attributs : val (valeur) et $nbpar$ (niveau maximal de parenthésage).

- $E' \rightarrow E$
- $E \rightarrow E + T \{ E(0).val = E(1).val + T.val ; E(0).nbpar = \max(E(1).nbpar, T.nbpar) \}$
- $E \rightarrow T \{ E.val = T.val ; E.nbpar = T.nbpar \}$
- $T \rightarrow (E) \{ T.val = E.val ; T.nbpar = E.nbpar + 1 \}$
- $T \rightarrow i \{ T.val = \text{lexical.val}(i) ; T.nbpar = 0 \}$

Pour $2+(3+4)$: le sous-arbre $(3+4)$ donne $val=7$ et $nbpar=0$; la règle $T \rightarrow (E)$ donne $T.val=7$, $T.nbpar=1$; puis $E.val=2+7=9$ et $E.nbpar=\max(0,1)=1$.

5.2 Arbre syntaxique décoré

Un **arbre syntaxique décoré** (ou arbre attribué) est un arbre syntaxique dont les nœuds portent en plus les valeurs de leurs attributs. Une DDS n'impose *aucun* ordre d'exécution des actions sémantiques, mais utiliser des attributs impose cependant un ordre implicite (on ne peut pas calculer un attribut avant ceux dont il dépend).

5.3 Attributs hérités et synthétisés

Type	Calculé à partir de...	Sens dans l'arbre	Grammaire associée
Synthétisé	attributs des non-terminaux de la partie droite (et éventuellement d'autres attributs du même nœud)	des feuilles vers la racine (bas $\rightarrow$ haut)	S-attribuée : uniquement des attributs synthétisés
Hérité	attributs du non-terminal de la partie gauche et éventuellement d'autres non-terminaux de la partie droite	de la racine vers les feuilles / entre frères (haut $\rightarrow$ bas)	L-attribuée : uniquement des attributs hérités ne dépendant pas des frères <i>droits</i>

- **Initialisation des synthétisés** : pour $X \rightarrow \alpha$ avec $\alpha \in V_T[\text{sup}][/\text{sup}]$ la valeur vient en général de l'analyseur lexical : $E \rightarrow i \{ E.val = \text{lexical.val}(i) \}$; pour $X \rightarrow \text{eps}$, c'est souvent l'élément neutre de l'opération (0 pour une somme, 1 pour un produit).
- **Initialisation des hérités** : l'axiome ne peut pas avoir d'attribut hérité (personne ne peut l'initialiser). On crée un nouvel axiome S' et la règle $S' \rightarrow S \{ S.nba = 0 \}$.

Exemple — compter les 'a' (synthétisé)

- $S \rightarrow aSb \{ S(0).nba = S(1).nba + 1 \} \cdot S \rightarrow SS \{ S(0).nba = S(1).nba + S(2).nba \} \cdot S \rightarrow \text{eps} \{ S(0).nba = 0 \}$

Pour le mot aabb : $S \rightarrow \text{eps}$ donne nba=0, puis aSb donne 1, puis encore aSb donne 2. La valeur « remonte » vers la racine.

5.4 Schéma de traduction dirigé par la syntaxe (STDS / TDS)

Un STDS est une DDS dans laquelle **l'ordre d'exécution des actions sémantiques est imposé** (les actions sont placées à des endroits précis dans la partie droite des règles).

Avec attribut hérité :	Avec attribut synthétisé :
$S' \rightarrow \{S.nba = 0;\} S \{ \text{afficher}("Fin"); \}$	$S' \rightarrow S \{ \text{afficher}(S.nba); \}$
$S \rightarrow a \{ S(1).nba = S(0).nba + 1; \} S$	$S \rightarrow aS \{ S(0).nba = S(1).nba + 1; \}$
$\quad \text{eps} \{ \text{afficher}(S.nba); \}$	$\quad \text{eps} \{ S.nba = 0; \}$

À retenir

Contre-exemple à éviter : utiliser une variable globale ($S' \rightarrow \{nba=0;\} S \{ \text{afficher}(nba); \} S \rightarrow a \{nba++;\} S \mid \text{eps}$). Ce n'est pas un attribut : on sort du formalisme des grammaires attribuées (pas de lien avec l'arbre, pas d'ordre garanti sur des évaluations quelconques).

5.5 Exemple : déclarations d'identificateurs

Grammaire $G : L \rightarrow T D ; T \rightarrow \text{int} \mid \text{float} ; D \rightarrow \text{id} , D \mid \text{id}$. Exemple de mot : int var, i. Objectif : construire la table des (identificateur, type).

Solution (1) : avec un attribut hérité

```
L -> T D      {D.typeh = T.type; L.table = D.table;}
T -> int      {T.type = "entier"}
T -> float    {T.type = "réel"}
D -> id , D    {D1.typeh = D0.typeh;
               D0.table = add((lexical.val(id), D1.typeh), D1.table);}
D -> id        {D.table = add((lexical.val(id), D.typeh), table_vide);}
```

Attributs : type (synthétisé de T), typeh (hérité de D : le type « descend » depuis T jusqu'aux identificateurs), table (synthétisé de D et L). Pour « int var, i » : $T.type = \text{entier} \rightarrow D.typeh = \text{entier}$ (hérité) $\rightarrow$ la table remonte : $\{(i, \text{entier}), (var, \text{entier})\}$.

Solution (2) : uniquement des attributs synthétisés (S-attribuée)

```
L -> T D      {L.table = MakeTable(T.type, D.collect);}
T -> int      {T.type = "entier"}      T -> float    {T.type = "réel"}
D -> id , D    {D0.collect = add(lexical.val(id), D1.collect);}
D -> id        {D.collect = add(lexical.val(id), coll_vide);}
```

Ici D ne collecte que les noms (collect = {i, var}) et c'est L qui, connaissant T.type, fabrique toute la table avec MakeTable(type, C). Avantage : évaluable pendant une analyse LR.

5.6 Graphe de dépendances

Définition

Graphe orienté dont les sommets sont les (nœud, attribut) ; il y a un arc de x vers y si le calcul de y dépend de celui de x.

- S'il contient un **cycle** : évaluation des attributs **impossible**.
- Sinon : il existe une énumération (tri topologique) des couples respectant les arcs ; c'est un ordre d'évaluation valide.

Exemple — « int var, i » avec la grammaire G' (solution 1)

Nœuds numérotés : [1] L, [2] T, [3] D (id var, D), [4] D (id i).

Ordre d'évaluation : ([2],type), ([3],typeh), ([4],typeh), ([4],table), ([3],table), ([1],table).

Lecture : T.type → D.typeh (nœud 3) → D.typeh (nœud 4) → table du nœud 4 → table du nœud 3 → table de L.

5.7 Évaluation des attributs pendant l'analyse syntaxique

- En toute généralité : arbre de dérivation + graphe de dépendances (coûteux en mémoire, ordre « quelconque »). Si l'analyse sémantique a lieu *après* la syntaxique : on peut le faire.
- Si elle a lieu *pendant* l'analyse syntaxique : on évite de construire l'arbre. Conditions : analyse **LR** → **grammaires S-attribuées** ; analyse **LL** → **grammaires L-attribuées**.

Cas LR(k) : on stocke les attributs dans la pile

Principe : on empile les valeurs d'attributs avec les non-terminaux. Lors d'une **réduction**, les attributs des symboles de la partie droite sont au sommet de la pile : on calcule l'attribut du non-terminal de gauche, puis on le remet dans la pile.

```
G1 :  E' -> E      {E'.val = E.val;}
      E  -> E + T   {E(0).val = E(1).val + T.val;}      E -> T   {E.val = T.val;}
      T  -> T * F   {T(0).val = T(1).val * F.val;}      T -> F   {T.val = F.val;}
      F  -> i      {F.val = lexical.val(i);}
```

Exemple — analyse de « 10+2*3 » avec calcul de la valeur

Entrée : i + i * i \$ (valeurs lexicales 10, 2, 3).

- Décaler i (10) → réduire F→i, T→F, E→T : pile « 0 E 1 », valeur 10.
- Décaler '+', décaler i (2), réduire F→i puis T→F : pile « 0 E 1 + 5 T 8 » (valeurs 10, 2).
- Lire '*': décaler (d6), décaler i (3), réduire F→i : pile « ... T 8 * 6 F 7 » (valeurs 10, 2, 3).
- Réduire T→T*F : T.val = 2*3 = 6. Réduire E→E+T : E.val = 10+6 = **16**. Accepter, E.val = 16.

Remarque : la priorité de * sur + est obtenue par la structure de la grammaire (T pour les produits, E pour les sommes).

Cas LL(k) : analyseur récursif avec attributs

Principe : les attributs **hérités** sont des paramètres d'entrée de la procédure et les **synthétisés** des valeurs de retour. Pour $X \rightarrow X_1 \dots X_k$: calculer les hérités de X_1 , appeler X_1 , puis calculer les hérités de X_2 (en fonction de ceux de X , et des synthétisés de X_1), etc., puis calculer les synthétisés de X . Un non-terminal peut donc utiliser les attributs de ses *frères gauches* (d'où « L-attribuée » : Left).

Exemple — grammaire LL pour les sommes, mot « 3+4 »

- $E \rightarrow T E' \{E'.h = T.s ; E.s = E'.s\}$
- $E' \rightarrow + T E' \{E'(1).h = E'(0).h + T.s ; E'(0).s = E'(1).s\}$
- $E' \rightarrow \text{eps} \{E'.s = E'.h\}$
- $T \rightarrow i \{T.s = \text{lexical.val}(i)\} \quad T \rightarrow (E) \{T.s = E.s\}$

s est synthétisé, h est hérité de E'. Exécution sur 3+4 : T.s = 3 ; E'.h = 3 (hérité) ; en lisant +T : T.s = 4 ; E'(1).h = 3+4 = 7 ; E'(1) → eps : E'(1).s = 7 ; remontée : E'(0).s = 7, E.s = **7**.

Pourquoi h ? La grammaire est récursive à droite (pour être LL) : on « accumule » la somme partielle dans l'attribut hérité h pendant la descente, puis on la renvoie.

5.8 Attributs avec JavaCup : attention au caractère LR

- En JavaCup, les actions sémantiques sont **toujours évaluées lors de réductions**.
- Une action placée au milieu d'une règle est donc **réécrite automatiquement** par JavaCup : on introduit un non-terminal vide qui porte l'action. Mais cela peut modifier le caractère LR de la grammaire !

Exemple — une action « au milieu » casse LR(1)

$G : S' \rightarrow S ; S \rightarrow \{\text{code1}\} a b ; S \rightarrow \{\text{code2}\} a c$. G est LR(0), donc LR(1).

Transformée $G' : S \rightarrow X a b ; X \rightarrow \text{eps} \{\text{code1}\} ; S \rightarrow Y a c ; Y \rightarrow \text{eps} \{\text{code2}\}$.

À l'état initial, il faut choisir entre réduire $X \rightarrow \text{eps}$ et $Y \rightarrow \text{eps}$ **avant** d'avoir lu quoi que ce soit d'utile : conflit réduction/réduction en LR(0), et en LR(1) aussi (les deux ont la prévision 'a'). G' n'est ni LR(0), ni SLR(1), ni LR(1), mais **LR(2)** (la 2^e lettre, b ou c, tranche).

5.9 Table des symboles

Lorsqu'un identificateur est rencontré (pour la première fois), il est ajouté dans la **table des symboles**. On la complète au fil de l'analyse avec : sa nature (classe, méthode, attribut, variable), son type (ou profil pour une méthode), son contrôle d'accès (private, protected, public), sa classe d'appartenance, son initialisation...

class A {	Ident	Nature	Type/Profil	Accès	Classe	Init	? Val
int var;	A	Classe		Package			
public boolean b = true;	var	Variable	int	Package	A	Non	
int m1 (A x) {...}	b	Variable	boolean	Public	A	Oui	true
private boolean n1 (A x, int n) {...}	m1	Méthode	A -> int	Package	A		
}	n1	Méthode	A, int -> boolean	Private	A		

- **À la déclaration** : on range l'identificateur dans la table ; s'il y était déjà → erreur « identificateur déjà défini ».
- **À l'utilisation** : on vérifie qu'il est présent dans la table, d'un type attendu (en tenant compte du sous-typage et de la coercion, i.e. cast implicite) et initialisé si nécessaire (membre droit d'une affectation). Sinon : erreur.

Portée des identificateurs : pile de tables des symboles

Il existe des portées : variables locales et paramètres d'une méthode, variables locales à un bloc, attributs d'une classe, classes locales... On gère cela avec une **pile de tables** :

- début d'une structure locale (bloc, méthode) → **on empile une table vide** ;
- fin de la structure → **on dépile** la table du sommet ;
- **ajout** : toujours dans la table du sommet ; **recherche** : on « descend » dans la pile jusqu'à trouver l'identificateur.

Exemple — portée

`void foo() { int x = 10; { char x = 'c'; } }` : valide en C (le x interne masque l'externe), **faux en Java** (redéclaration d'une variable locale dans la portée englobante).

`class essai { char y = 'a'; boolean foo(char x) { char z = 'b'; return (x == y); } }` : dans `foo`, la table du sommet contient `z` et `x` ; la table de la classe (en dessous) contient `foo` et `y`. Pour `x == y`, `x` est trouvé dans la table du sommet, `y` en descendant dans la pile.

5.10 Vérification de types (typage statique)

- **Typage statique** : vérifié à la compilation. **Typage dynamique** : vérifié seulement à l'exécution.
- On vérifie notamment : une méthode/un attribut appartient bien à la classe de l'objet sur lequel il s'applique ; une méthode est appelée avec des arguments du bon type ; membre gauche et droit d'une affectation ont le même type (ou compatible) ; etc.

Pourquoi c'est souvent complexe :

- **Sous-typage et conversion implicite** : en C on peut affecter un char à un int, faire de l'arithmétique sur les pointeurs ; en Java, 10 est vu selon le contexte comme un entier ou un flottant.
- **Héritage** : un objet de la classe `c'` (qui hérite de `c`) est aussi un objet de `c` ; une méthode appelée sur un objet de `c'` peut être définie dans `c`.

- **Surcharge** (Java, Ada) : un même identificateur peut désigner des méthodes différentes (choix selon les types des arguments).
- **Polymorphisme** (ML) et **généricité** (Ada) : le type d'une fonction dépend de l'instanciation ou de l'utilisation.

À retenir

L'analyse sémantique = vérification de propriétés statiques contextuelles ; on l'implante en appelant des procédures d'évaluation d'attributs dans l'algorithme d'analyse syntaxique, selon un schéma de traduction.

LR $\leftrightarrow$ grammaire S-attribuée ; LL $\leftrightarrow$ grammaire L-attribuée.

6. Production de code

6.1 Vue d'ensemble

```
Arbre syntaxique décoré (ou abstrait)
  ↓ Génération du code intermédiaire
Code intermédiaire (code à 3 adresses)
  ↓ Optimisations indépendantes de la machine
Code intermédiaire (optimisé)
  ↓ Allocation / assignement des registres
  ↓ Génération du code en langage cible
Code en langage cible
  ↓ Optimisations dépendantes de la machine
Programme en langage cible
```

6.2 Arbre de syntaxe abstraite et simplification

La production de code intermédiaire s'appuie sur l'**arbre de syntaxe abstraite (AST)** construit lors de l'analyse syntaxique : une version épurée de l'arbre de dérivation qui garde l'essentiel (par exemple la règle `Inst → si Cond alors Inst sinon Inst` devient un nœud `Si_alors_sinon(C, I1, I2)`).

Simplification par réécriture

```
Si_alors_sinon(vrai, x1, x2) -> x1      Mult(x1, 0) -> 0      Mult(x1, 1) -> x1
Si_alors_sinon(faux, x1, x2) -> x2      Mult(0, x1) -> 0      Mult(1, x1) -> x1
```

Simplification par évaluation partielle

On remplace par sa valeur une expression calculable à la compilation (entiers, réels, constantes combinés par des opérateurs prédéfinis). Exemple : `x := 3 + 7` devient `x := 10` (l'AST `affect(x, +(3,7))` devient `affect(x, 10)`).

6.3 Environnement d'exécution : organisation de la mémoire

Questions côté langage source : fonctions récursives ? passage de paramètres (appel) ? allocation mémoire dynamique (et libération explicite ?). Côté machine cible : longueur des mots/adresses, entités directement adressables, instructions d'accès spécifiques... Modèle général de la mémoire :

Zone	Contenu
Code cible	le programme compilé
Données statiques	variables globales, constantes
Pile de contrôle	appels de fonctions (paramètres, variables locales, adresse de retour)
Tas	allocation dynamique

6.4 Code intermédiaire : le code à 3 adresses

Le code intermédiaire est exprimé en termes de variables (en nombre illimité), registres, transferts, branchements et entrées/sorties. Il contient des variables de type de base (entier, booléen, flottant), des opérations mémoire explicites, des expressions simples (avec variables temporaires), des sauts conditionnels ou non, des CALL/RET.

```
x := y op z      (affectation binaire)      aller en (a)      (saut inconditionnel)
x := opu y      (affectation unaire)      si y op_rel z alors aller en (a)      (saut conditionnel)
x := y          (copie)                    Lire x      Écrire y
x[i] := y       (affectation indicée)
```

Exemple — traduction

```
x := 3*y + z ; y := y*x devient : (1) t := 3*y (2) x := t+z (3) y := y*x
while x != 0 do y[x] := z ; x := x-1 endwhile devient :
(1) si x = 0 aller en (5) (2) y[x] := z (3) x := x-1 (4) aller en (1) (5) ...
```

Génération pour les expressions booléennes

- **Évaluation systématique** : tous les termes sont évalués.
- **Évaluation court-circuit** : seuls les termes nécessaires sont évalués.

Exemple — $x := \text{vrai ou } (y \text{ et } z)$

Systématique : (1) $t := y \text{ et } z$ (2) $x := \text{vrai ou } t$

Court-circuit : (1) $x := \text{vrai}$ (puisque « vrai ou ... » est toujours vrai, y et z ne sont pas évalués).

La génération se fait par **traduction dirigée par la syntaxe** avec une grammaire attribuée (attributs `code`, `reg`, `inst_suiv`) ; il faut gérer branchements et étiquettes pour les instructions.

6.5 Optimisations du code intermédiaire (indépendantes de la machine)

Elles reposent sur une **analyse statique du flot de données / de contrôle** : variables « constantes » (évaluation, réduction et propagation de constantes), code mort, code redondant, expressions calculées plusieurs fois sans redéfinition de leurs composantes (**élimination des sous-expressions communes**), propagation des copies, extraction des **invariants de boucle**...

a) Élimination des sous-expressions communes

Exemple — correct vs incorrect

Code : (1) $a := b+c$ (2) $b := a-d$ (3) $c := b+c$ (4) $d := a-d$

- (4) $d := a-d$ **peut** devenir $d := b$ (a et d n'ont pas changé entre (2) et (4)). **Correct.**
- (3) $c := b+c$ **ne peut pas** devenir $c := a$: entre (1) et (3), b a été redéfini en (2), donc $b+c$ n'a plus la même valeur. **Incorrect !**

Autre piège : (1) $x := a+1$ (2) $y := a+1$... (10) $a := 10$ (11) aller en (2). Remplacer (2) par $y := x$ est correct à la première exécution mais **faux** après le saut en (2), car a a changé. D'où le besoin d'informations sur le **flot de contrôle** (et pas seulement sur l'ordre du code).

b) Graphe de flot de contrôle

Définition

Graphe dont les sommets sont des **blocs séquentiels** de code intermédiaire ; il y a un arc $B1 \rightarrow B2$ si l'exécution de $B1$ peut être suivie de celle de $B2$: (i) si $B1$ se termine par un branchement (conditionnel ou non) vers $B2$; (ii) si $B2$ suit $B1$ dans le code et $B1$ ne se termine pas par un branchement inconditionnel.

Tête de bloc : (1) la première instruction du programme ; (2) une instruction suivant un branchement ; (3) une instruction « visée » par un branchement. Un bloc = de la tête jusqu'à l'instruction précédant la tête suivante.

Exemple — construction du graphe

(1) $i:=3$ (2) $t1:=4*i$ (3) $t2:=a[i]$ (4) $j:=2$ (5) $j:=j+1$ (6) si $j>100$ aller en (12) (7) si $t2<b$ aller en (10) (8) $t2:=t2+3$ (9) aller en (7) (10) $b:=b-j$ (11) aller en (5) (12) $a[t1]:=t2$

Têtes : 1 (début), 5 (cible de « aller en 5 »), 7 (cible), 8 (suit un branchement), 10 (cible), 12 (cible).

Blocs : $B1=\{1,2,3,4\}$, $B2=\{5,6\}$, $B3=\{7\}$, $B4=\{8,9\}$, $B5=\{10,11\}$, $B6=\{12\}$.

Arcs : $B1 \rightarrow B2$; $B2 \rightarrow B3$ (chute) et $B2 \rightarrow B6$ (saut en 12) ; $B3 \rightarrow B4$ (chute) et $B3 \rightarrow B5$ (saut en 10) ; $B4 \rightarrow B3$ (aller en 7) ; $B5 \rightarrow B2$ (aller en 5).

c) Optimisation locale (intra-bloc) et globale (inter-blocs)

- **Locale** : interne à un bloc du graphe de flot.
- **Globale** : traite en même temps plusieurs blocs ; nécessite des informations complémentaires sur le flot de données.

Algorithme local de sous-expressions communes dans un bloc B : on maintient `Disp`, l'ensemble des couples (expression, variable) disponibles. Pour chaque instruction (i) de type $x := op$ y ou $x := y op z$: si `Disp` contient un couple $(op\ y, var) \rightarrow$ on remplace par $x := var$, puis (si $x \neq var$)

on ôte de Disp tout couple (exp, x) ; sinon on ajoute (op y, x) à Disp. Enfin, on ôte de Disp tout couple (expr, v) tel que x apparaît dans expr (x vient d'être redéfini).

Exemple — Disp au fil d'un bloc

- (1) $x := b+c \rightarrow \text{Disp} = \{(b+c, x)\}$
- (2) $y := b+c \rightarrow (b+c)$ est dans Disp : remplacé par $y := x$; Disp inchangé
- (3) $z := y+2 \rightarrow \text{Disp} = \{(b+c, x), (y+2, z)\}$
- (i) $t := b+c \rightarrow$ remplacé par $t := x$
- (j) $z := x+3 \rightarrow z$ redéfini : on retire $(y+2, z)$; $z := x+3$ ajouté

Versión globale : pour chaque bloc B on calcule des ensembles d'expressions : **Prod(B)** (produites : calculées dans B et dont les opérandes ne sont pas redéfinis ensuite dans B), **Supp(B)** (supprimées : expressions dont un opérande est redéfini dans B sans être recalculée après), **In(B)** et **Ex(B)** (disponibles en entrée / en sortie de B).

```
Prod(B) = {} ; pour chaque instruction (i) de B de type  $x := \dots$  :  
  si (i) est de type  $x := \text{op } y$  ou  $x := y \text{ op } z$  : ajouter op y (ou y op z) à Prod(B)  
  ôter de Prod(B) toute expression contenant x  
  
Supp(B) = {} ; E = ensemble de toutes les expressions du programme  
  pour chaque instruction de B de type  $x := \dots$  :  
    ajouter à Supp(B) toutes les expressions de E contenant x et non recalculées plus loin dans B  
  
Ex(B) = (In(B) - Supp(B))  $\cup$  Prod(B)      In(B) =  $\bigcup$  Ex(P) pour tous les prédécesseurs P de B  
(B1 = bloc d'entrée : In(B1) = {}, Ex(B1) = Prod(B1) ; on répète jusqu'à stabilité des ensembles)
```

Idée de l'intersection : une expression n'est disponible en entrée de B que si elle l'est sur *tous* les chemins qui y mènent. **Utilisation** : pour une instruction $x := \text{op } y$ (ou $y \text{ op } z$) du bloc B, si op y est disponible à l'entrée de B et si ni y ni z ne sont modifiés avant (i) dans B, alors on introduit une nouvelle variable u : à la dernière évaluation de l'expression sur les chemins arrivants on ajoute $u := \text{op } y$, et on remplace (i) par $x := u$. Les passes sont **itératives** : plusieurs passes successives sont nécessaires.

d) Propagation des copies

Une copie (i) $x := y$ peut être éliminée en remplaçant x par y dans toute utilisation de x en un point p, si pour tout chemin : (i) est l'unique définition de x atteignant p, et aucun chemin de (i) à p ne redéfinit y (attention aux boucles !).

Exemple — propagation des copies

Avant : (1) $x := y$ (2) $z := x+1$ (3) $w := 2*z$ (4) $x := x-1$ (5) $t := x+w$

Après : (1) $z := y+1$ (2) $w := 2*z$ (3) $x := y-1$ (4) $t := x+w$

$x := y$ est propagée aux utilisations de x tant que x et y ne changent pas ; en (4), x est redéfini, donc dans $t := x+w$ on garde x. Comme pour les sous-expressions communes, cela se fait en intra-bloc mais aussi globalement.

e) Invariants de boucle

Exemple — sortir un calcul de la boucle

Avant (dans la boucle) : $x := y+z$; $w := x+10$; $a := a+w$; $i := a+1$

$x := y+z$ et $w := x+10$ ne dépendent d'aucune variable modifiée dans la boucle : ils sont

invariants. On les sort avant la boucle ; la boucle ne garde que $a := a+w$; $i := a+1$. Le calcul n'est plus refait à chaque itération.

6.6 Production du code cible : machine abstraite → machine concrète

But : générer du langage machine ou d'assemblage pour une machine disposant d'un nombre fini de registres, d'une mémoire de taille de mots fixée, d'une unité centrale (jeu d'instructions, modes d'adressage, unité arithmétique et logique) et d'appels système. Trois problèmes liés les uns aux

autres (et dépendants de la machine) : **assigner les variables** à la mémoire (RAM ou registres), **allouer les registres**, **transformer chaque instruction** intermédiaire en instruction(s) assembleur.

Assignement / allocation des registres

- Problème : beaucoup de variables dans le code intermédiaire, peu de registres. Certaines instructions exigent des registres (opérandes arithmétiques, adressage indexé...). Un calcul entre deux registres est plus rapide qu'entre un registre et la mémoire.
- Le code produit doit choisir quelle variable placer dans quel registre à chaque point, charger les données au bon moment et libérer les registres en remplaçant des données en mémoire si nécessaire.

Méthode par coloriage de graphe : on construit un **graphe d'interférence GI** : les sommets sont les variables ; il y a une arête entre a et b si a est définie à une instruction où b est vivante (b utilisée plus tard). Deux variables qui interfèrent ne peuvent pas être dans le même registre. On colorie GI avec **k couleurs** (k registres) : deux sommets reliés n'ont pas la même couleur. Si on y arrive avec $\leq k$ couleurs, on a un assignement ; sinon on insère du code de chargement/déchargement (« spill » en mémoire).

Exemple — coloriage

(1) $a := 1$ (2) $b := 2$ (3) $c := a + b$ (4) $d := c * 2$ (5) écrire d

Interférences : a et b sont vivantes en même temps (a est encore utilisée en (3) quand b est définie) : arête a—b. Après (3), a et b ne servent plus : c n'interfère avec aucune des deux ; d non plus.

Avec $k = 2$: $a = R0$, $b = R1$, $c = R0$, $d = R0$. Deux registres suffisent pour quatre variables.

Génération du code : choisir les instructions les plus efficaces

Code intermédiaire :	Assembleur basique :	Avec INC :	Avec adressage indexé :
(1) $i := 0$	MOV R0, t	MOV R0, t	MOV R0, t
(2) si $i=100$ aller en (6)	MOV R1, 0	MOV R1, 0	MOV RI, 0
(3) $t(i) := 0$	Debut : COMP R1,100	Debut: COMP R1,100	Debut: COMP RI,100
(4) $i := i+1$	JEQ Suite	JEQ Suite	JEQ Suite
(5) aller en (2)	MOV [R0], 0	MOV [R0], 0	MOV [R0+RI], 0
(6) ...	ADD R0, 1	INC R0	INC RI
	ADD R1, 1	INC R1	JMP Debut
	JMP Debut	JMP Debut	Suite : ...
	Suite : ...	Suite : ...	

Le même code intermédiaire peut être traduit de plusieurs façons : l'instruction INC remplace ADD x,1, et l'adressage indexé évite de tenir à jour deux registres (R0 pour l'adresse et R1 pour l'indice), d'où un code plus court et plus rapide.

Optimisations du code cible (dépendantes de la machine)

- **Inlining** (développement en ligne) : on remplace l'appel CALL/RET à un sous-programme par le corps du sous-programme. On perd de la place (donc à réserver aux « petites » fonctions) mais on gagne en temps.
- Prise en compte des modes d'adressage ; exploitation des possibilités de parallélisme du processeur cible ; etc.

À retenir

Production de code intermédiaire naïf : facile avec une grammaire attribuée. Optimisations : compromis coût/efficacité à étudier ; un compilateur donné n'implante pas forcément toutes les optimisations.

Analyses lexicale, syntaxique et sémantique : environ **10 %** du temps de compilation. Génération et optimisation du code : environ **90 %**.

Le code intermédiaire permet de « porter » le compilateur vers une autre machine à moindre coût (seul le back-end change).

Lien avec les TP : création d'analyseurs LR (LALR) avec JFlex et JCup.

7. Récapitulatif et pièges classiques

7.1 Tableau de synthèse

Notion	À savoir en une phrase
Grammaire augmentée	On ajoute $S' \rightarrow S$ pour avoir un unique point d'acceptation.
Conflit décalage/réduction	Dans un état, on peut à la fois lire un terminal (décaler) et réduire sur ce terminal.
Conflit réduction/réduction	Deux réductions possibles sur le même terminal dans le même état.
SLR(1)	Réduit sur Suivant(X) : prévision globale, assez grossière.
LR(1)	Items $X \rightarrow \alpha.\beta$, a : prévision par contexte ; beaucoup d'états.
Saturation	Ajouter les règles de Y quand le point précède Y, avec prévisions Premier(γa).
LALR(1)	LR(1) avec fusion des états de même cœur LR(0) : autant d'états que LR(0).
Mode panique	Pas de correction ; on se resynchronise (continuation ou restart).
Syntagme / local / global	Routines dans la table / règles d'erreur dans la grammaire / devine l'intention (inutilisable).
DDS / STDS	Grammaire + règles sémantiques / même chose avec l'ordre des actions imposé.
Synthétisé / hérité	Bas → haut (S-attribuée, LR) / haut → bas et frères gauches (L-attribuée, LL).
Graphe de dépendances	Un cycle rend l'évaluation impossible ; sinon tri topologique = ordre d'évaluation.
Table des symboles	Nature, type, accès, classe, init ; pile de tables pour la portée.
Code à 3 adresses	$x := y \text{ op } z$, copies, sauts, CALL/RET : proche de l'assembleur, indépendant de la machine.
Bloc de base / graphe de flot	Séquence sans branchement interne ; arcs = successions possibles.
Prod / Supp / In / Ex	Pour les sous-expressions communes globales : $\text{In}(B) = n \text{ Ex}(\text{préd.})$, $\text{Ex}(B) = (\text{In} - \text{Supp}) \cup \text{Prod}$.
Coloriage de graphe	Variables en interférence → registres différents ; k couleurs = k registres.

7.2 Pièges classiques

- Oublier que la prévision d'un item hérité par saturation est **Premier(γa)**, pas seulement a.
- Confondre SLR(1) et LR(1) : en SLR on réduit sur Suivant(X) entier, en LR(1) seulement sur la prévision de l'item.
- Croire que LALR(1) ajoute des conflits décalage/réduction : il ne peut ajouter que des conflits **réduction/réduction**.
- Utiliser des attributs hérités avec un analyseur LR : il faut une grammaire S-attribuée (ou la transformer).
- Mettre une action « au milieu » d'une règle avec JavaCup sans penser à la réécriture (non-terminal vide) qui peut rendre la grammaire non LR(1).
- Utiliser une variable globale à la place d'un attribut dans un schéma de traduction.
- Appliquer l'élimination des sous-expressions communes sans vérifier que les opérandes n'ont pas été redéfinis (et sans tenir compte du flot de contrôle, boucles comprises).
- Oublier qu'en Java on ne peut pas redéclarer une variable locale dans un bloc interne (contrairement à C).